

2.1.3. Pertanyaan

1. Pada konstruktor, mengapa nilai awal atribut front dan rear bernilai -1, sementara atribut size bernilai 0?

front = -1 dan rear = -1: Digunakan sebagai penanda (flag) bahwa antrian masih dalam kondisi kosong. Karena indeks array di Java dimulai dari 0, nilai -1 menunjukkan bahwa belum ada data yang menempati posisi depan maupun belakang di dalam array.

size = 0: Karena jumlah elemen atau data yang tersimpan di dalam antrian saat pertama kali dibuat memang masih nol (belum ada elemen yang dimasukkan).

2. Pada method Enqueue, jelaskan maksud dan kegunaan dari potongan kode berikut!

```
if (rear == max - 1) {  
  
    rear = 0;
```

Potongan kode ini digunakan untuk menerapkan konsep **Circular Queue**

- Jika posisi rear (penunjuk elemen paling belakang) sudah mencapai batas akhir indeks array (max - 1), namun antrian tersebut sebenarnya belum penuh (karena sudah ada elemen di depan yang dikeluarkan lewat operasi Dequeue), maka posisi rear akan diputar kembali ke indeks 0.
- **Kegunaan:** Memaksimalkan penggunaan memori array. Tanpa kondisi ini, space array yang kosong di bagian depan tidak akan bisa digunakan lagi setelah rear mencapai ujung array.

3. Pada method Dequeue, jelaskan maksud dan kegunaan dari potongan kode berikut!

```
if (front == max - 1) {  
  
    front = 0;
```

- Ketika proses pengambilan data (*Dequeue*) dilakukan dan penunjuk front (elemen terdepan) sudah berada di ujung akhir array (max - 1), maka setelah data tersebut diambil, posisi front akan **bergeser atau berputar kembali ke indeks 0**.
- **Kegunaan:** Memastikan penunjuk front dapat terus mengikuti perputaran data di dalam array melingkar secara tepat saat elemen demi elemen dihapus.

4. Pada method print, mengapa pada proses perulangan variabel i tidak dimulai dari 0 (int i=0), melainkan int i=front?

Karena posisi elemen terdepan di dalam *Queue* tidak selalu berada di indeks 0. Akibat proses *Enqueue* dan *Dequeue* yang dinamis (terutama pada *Circular Queue*), elemen terdepan yang sah saat itu ditunjukkan oleh nilai atribut front. Jika perulangan dimulai dari 0, data yang dicetak berpotensi salah (mencetak data lama yang sudah di-*dequeue* atau urutannya tidak sesuai antrian)

5. Perhatikan kembali method print, jelaskan maksud dari potongan kode berikut!

```
i = (i + 1) % max;
```

Potongan kode ini berfungsi untuk menggeser indeks *i* ke elemen berikutnya secara melingkar.

Operasi *modulus* (% max) memastikan bahwa ketika nilai *i* + 1 sama dengan nilai max (melampaui indeks terakhir array), nilai *i* akan otomatis kembali menjadi 0. Ini mencegah terjadinya `ArrayIndexOutOfBoundsException` dan memastikan semua elemen dari front hingga rear tercetak dengan benar meski posisinya berputar.

6. Tunjukkan potongan kode program yang merupakan queue overflow!

```
if (IsFull()) {  
    System.out.println(x: "Queue sudah penuh");  
}
```

7. Pada saat terjadi queue overflow dan queue underflow, program tersebut tetap dapat berjalan dan hanya menampilkan teks informasi. Lakukan modifikasi program sehingga pada saat terjadi queue overflow dan queue underflow, program dihentikan!

```
public int Dequeue() {  
    int dt = 0;  
  
    if (IsEmpty()) {  
        System.out.println(x: "Queue Underflow! Program dihentikan.");  
        System.exit(status: 0);  
    } else {  
        dt = data[front];  
        size--;  
  
        if (IsEmpty()) {  
            front = rear = -1;  
        } else {  
            if (front == max - 1) {  
                front = 0;  
            } else {  
                front++;  
            }  
        }  
    }  
}
```

```

public void Enqueue(int dt) {
    if (IsFull()) {
        System.out.println(x: "Queue Overflow! Program dihentikan.");
        System.exit(status: 0);
    } else {
        if (IsEmpty()) {
            front = rear = 0;
        } else {
            if (rear == max - 1) {
                rear = 0;
            } else {
                rear++;
            }
        }
        data[rear] = dt;
        size++;
    }
}
}

```

2.2.3 Pertanyaan

Lakukan modifikasi program dengan menambahkan method baru bernama LihatAkhir pada class AntrianLayanan yang digunakan untuk mengecek antrian yang berada di posisi belakang. Tambahkan pula daftar menu 6. Cek Antrian paling belakang pada class LayananAkademikSIAKAD sehingga method LihatAkhir dapat dipanggil!

```

// modifikasi 2.2.3
case 6:
    antrian.lihatAkhir();
    break;

case 0:
    System.out.println(x: "Terima kasih.");
    break;

default:
    System.out.println(x: "Pilihan tidak valid.");
}
} while (pilihan != 0);

```

```

// modifikasi 2.2.3
System.out.println(x: "6. Cek Antrian Paling Belakang");

```

```

// modifikasi 2.2.3
public void lihatAkhir() {
    if (isEmpty()) {
        System.out.println(x: "Antrian kosong.");
    } else {
        System.out.print(s: "Mahasiswa paling belakang: ");
        System.out.println(x: "NIM - NAMA - PRODI - KELAS");
        data[rear].tampilkanData();
    }
}

```